

Guide de bonnes pratiques R

Migration des programmes SAS vers R

Version 1.0 • 27/12/2025

Objectif : standardiser le codage en R, sécuriser la migration SAS -> R, et faciliter l'appropriation par les équipes métier.

Public cible : développeurs/analystes, équipe QA/recette, data management, et référents métier.

Périmètre : scripts R (batch) et notebooks (Quarto/R Markdown) utilisés pour produire des datasets, indicateurs, contrôles et livrables.

1. Principes directeurs

La migration SAS -> R ne consiste pas à “réécrire ligne à ligne”, mais à conserver les mêmes règles métier et contrôles, tout en adoptant des standards R qui améliorent la lisibilité, la maintenabilité et l’auditabilité.

1.1. Les 6 principes à respecter

- Lisibilité : un code compréhensible en 5 minutes par un pair.
- Reproductibilité : mêmes résultats à partir des mêmes entrées, sur poste ou serveur.
- Traçabilité : entrées, sorties, paramètres, versions et logs clairement identifiés.
- Robustesse : erreurs gérées, contrôles qualité systématiques, valeurs manquantes maîtrisées.
- Modularité : fonctions réutilisables, peu de copier-coller, séparation “I/O - logique - reporting”.
- Performance pragmatique : optimiser quand c’est nécessaire, pas avant.

1.2. Ce qu’on standardise (et ce qu’on laisse libre)

Standard obligatoire	Standard recommandé	Libre / selon contexte
Structure de projet, conventions de nommage, gestion des dépendances, logs, contrôles QA, format des sorties	Style guide, outillage tests/CI, packaging interne, patterns de performance	Choix tidyverse vs data.table, approche Quarto vs scripts batch, outils BI aval

2. Organisation d’un projet R (standard de repository)

Adopter une structure stable facilite la migration, la recette et l’exploitation.

2.1. Arborescence type

Recommandation (adaptable selon contraintes) :

```
project/  
  README.md  
  renv.lock  
  R/  
    00_config.R  
    10_io_import.R  
    20_transformations.R  
    30_controles_qa.R  
    40_exports.R  
  data/  
    raw/      # données sources (lecture seule)  
    ref/      # référentiels  
    work/     # données intermédiaires (recréables)  
    out/      # livrables (datasets, fichiers)  
  tests/  
  docs/      # dictionnaires, règles métier, spécifs  
  logs/
```

2.2. Reproductibilité des dépendances

- Utiliser renv (ou équivalent) pour figer les versions des packages ; versionner le fichier renv.lock.
- Documenter la version de R (ex. 4.3.x) et la plateforme cible (Windows/Linux).
- Éviter les installations “à la main” non tracées ; privilégier un bootstrap script (ex. scripts/setup.R).

2.3. Paramétrage et exécution

- Paramètres d'exécution dans un fichier unique (YAML/JSON) plutôt que des variables dispersées.
- Scripts batch appelables en ligne de commande (Rscript) ; éviter les chemins absolus.
- Centraliser les chemins avec here ou un helper interne (ex. path_*).

3. Conventions de style et lisibilité

3.1. Nommage (variables, fonctions, objets)

- Choisir une convention et s'y tenir : snake_case recommandé (ex. date_traitement, montant_ht).
- Préfixer les objets par leur rôle si utile : df_* (data frame), tbl_* (table), vec_* (vecteur), cfg_* (config).
- Nommer les fonctions avec des verbes : calculer_score(), charger_sources(), exporter_livvable().
- Éviter les noms trop courts ou ambigus (x, tmp) sauf dans des contextes très locaux (1-2 lignes).

3.2. Structure d'un script standard

1. En-tête : but, auteur, date, version, entrées, sorties, dépendances, tickets de changement.
2. Chargement des packages (regroupé), options, seed, locale si nécessaire.
3. Lecture des entrées + contrôle “existence / schéma / types”.
4. Transformations (logique métier) sous forme de fonctions.
5. Contrôles QA (comptages, clés, bornes, duplicats, totaux).
6. Exports + résumé de fin (volumétrie, emplacement des sorties).

3.3. Formatage et règles simples

- Indentation 2 espaces ; pas de tabulations.
- Longueur de ligne cible 100-120 caractères (adapter si besoin).
- Un objet = une responsabilité ; préférer plusieurs fonctions courtes à une fonction “géante”.
- Commenter le “pourquoi”, pas le “quoi”. Les règles métier doivent être référencées (spec).
- Utiliser un formateur automatique (styler) et un linter (lintr) dans le pipeline.

4. Données : import, types, NA, encodages

4.1. Import/export robustes

- Pour CSV/TSV : spécifier l'encodage (UTF-8), le séparateur et les NA ; conserver un dictionnaire de colonnes.
- Pour SAS7BDAT : privilégier haven ; contrôler les formats, labels et dates.
- Pour gros volumes : considérer Arrow/Parquet ; limiter les I/O répétées en cache contrôlé.

4.2. Types et dates (point critique en migration)

- Normaliser les dates dès l'import (Date / POSIXct). Documenter les fuseaux si nécessaires.
- Gérer explicitement les facteurs : décider si une variable doit rester caractère ou devenir facteur.

- Ne jamais laisser une colonne “deviner” son type sans contrôle (risque de régression silencieuse).

4.3. Valeurs manquantes et conventions

- R utilise NA ; SAS utilise . (numérique) et " (caractère). Définir des règles de conversion.
- Toujours expliciter le traitement des NA dans les agrégations (na.rm=TRUE) et le tracer.
- Maintenir un tableau de correspondance des codes manquants métier (ex. -9, 9999) et les recoder.

5. Transposition SAS -> R : patterns recommandés

5.1. Équivalences courantes (mémo)

SAS (concept)	R (approche)	Remarques migration
DATA step (set, merge, by)	dplyr (mutate, left_join) ou data.table	Attention aux doublons et à la logique de merge (cartésien). Contrôler les clés.
PROC SQL	dplyr + dbplyr (si base) ou DBI	Si les données sont en base, pousser les calculs côté SQL via dbplyr.
PROC SORT nodupkey	distinct() / arrange() / unique()	Définir la règle de déduplication (clé, priorité, date max, etc.).
Formats/Informats	dictionnaires + fonctions de recodage	Formaliser les tables de mapping et les versionner.
Macro variables & %macro	fonctions R + fichiers de config	Éviter l'évaluation dynamique quand possible ; préférer des fonctions pures.
Libname	chemins + connecteurs DB (DBI)	Standardiser la gestion des chemins et connexions (variables d'environnement).
ODS / PROC REPORT	Quarto/R Markdown + gt/flextable	Séparer calcul et rendu ; garantir une sortie reproductible.

5.2. “DATA step mindset” en R

- Penser en pipelines : lecture -> transformation -> contrôles -> export.
- Une transformation = une étape testable (préférer plusieurs étapes explicites à une étape “magique”).
- Éviter les effets de bord (modification d'objets globaux) : tout doit dépendre des entrées.

5.3. Contrôles qualité inspirés SAS (indispensables)

- Contrôle volumétrie : lignes avant/après chaque étape clé, taux de perte/duplication.
- Contrôle clés : unicité, non-nullité, intégrité référentielle (joins).
- Contrôle bornes : min/max, valeurs interdites, distributions (par segment si pertinent).
- Contrôle agrégats : totaux et sous-totaux comparés au “golden run” SAS.

6. Modularisation : fonctions, packages, réutilisation

6.1. Règles

- Extraire les règles métier en fonctions paramétrées plutôt que copier-coller entre scripts.
- Créer un package interne quand un ensemble de fonctions est partagé entre projets.

- Respecter une interface simple : inputs explicites, outputs explicites, erreurs explicites.

6.2. Gestion des paramètres

- Centraliser les paramètres dans cfg (liste) issu d'un fichier YAML/JSON.
- Valider la configuration au démarrage (types, présence des champs) et échouer tôt.
- Loguer la configuration effective (hors secrets).

7. Logs, erreurs et auditabilité

7.1. Logging

- Un log par exécution (horodaté), avec : version du code, version de R, packages, paramètres, volumétrie, durée.
- Niveaux : INFO (étapes), WARN (écarts tolérés), ERROR (bloquant).
- Écrire les logs dans un dossier dédié et/ou vers un collecteur (selon SI).

7.2. Gestion d'erreurs

- Utiliser tryCatch pour capturer et enrichir les erreurs aux frontières I/O.
- Échouer tôt : si un input manque ou un schéma est incorrect, arrêter immédiatement.
- Message d'erreur orienté métier : quelle règle a échoué, sur quel périmètre, quel impact.

8. Tests, recette et stratégie de validation SAS vs R

8.1. Niveaux de tests

- Unitaires : tester une fonction sur des cas simples et des cas limites.
- Intégration : tester un pipeline complet sur un échantillon figé.
- Non-régression : rejouer les scénarios clés et comparer aux résultats attendus.

8.2. Stratégie de comparaison avec SAS

7. Définir un jeu de données de référence (golden dataset) et des sorties attendues.
8. Comparer : nombres de lignes, clés, agrégats, distributions, et fichiers produits.
9. Tolérances documentées : arrondis, formats date, ordres de tri, encodage.
10. Tracer toute divergence : cause, décision (corriger / accepter), validation métier.

8.3. Gestion des arrondis et formats

- SAS et R peuvent différer sur certains arrondis / représentations : fixer une règle (ex. arrondi bancaire).
- Standardiser les formats d'export (CSV UTF-8, Parquet, Excel) et les règles de nommage.

9. Performance et industrialisation

9.1. Règles de base

- Éviter les boucles R sur gros volumes ; préférer dplyr/data.table/vectorisation.
- Limiter les copies : sélectionner les colonnes utiles, filtrer tôt, agréger au plus tôt.
- Mesurer avant d'optimiser (profvis, bench) ; documenter les optimisations.

9.2. Choix d'outils selon volumétrie

Cas	Recommandation	Notes
< 5M lignes	dplyr + readr	Suffisant dans la plupart des cas ; maintenir la lisibilité.
5M à 50M lignes	data.table ou Arrow/Parquet	Attention mémoire ; privilégier Parquet pour échanges.
> 50M lignes / en base	dbplyr + SQL / traitements côté base	Pousser les calculs au plus proche des données.

10. Documentation, revue de code et gouvernance

10.1. Documentation minimale attendue

- README : contexte, prérequis, comment exécuter, entrées/sorties, structure du projet.
- Dictionnaire de données : colonnes, types, règles, sources, fréquences de mise à jour.
- Changelog : ce qui a changé, pourquoi, impact sur les résultats.

10.2. Revue de code (checklist)

- Nommage et lisibilité conformes au standard.
- Contrôles QA présents et pertinents.
- Gestion des NA explicite.
- Pas de chemins absolus, secrets ou identifiants en clair.
- Performance raisonnable et I/O maîtrisées.
- Tests présents (au minimum sur les règles sensibles).

10.3. CI/CD (si possible)

- Lint + format automatique.
- Exécution des tests.
- Build d'un artefact (zip, package, rapport) versionné.
- Déploiement reproductible (conteneur ou environnements figés).

11. Accompagnement des équipes métier (adoption)

11.1. Stratégie d'appropriation

- Mettre en place un glossaire SAS -> R (proc, macro, formats, notions).
- Former sur des cas réels : une procédure SAS migrée bout en bout avec contrôle.
- Standardiser les sorties métier : mêmes colonnes, mêmes libellés, mêmes contrôles.
- Organiser des "recettes conjointes" : métier valide l'équivalence, pas seulement l'IT.

11.2. Artefacts utiles pour le métier

- Tableau de correspondance des indicateurs (définition, source, règle, script R associé).
- Rapports Quarto/R Markdown reproductibles (avec paramètres).
- Fiches de contrôle (écarts, tolérances, décisions).

12. Modèles et gabarits recommandés

12.1. Gabarit d'en-tête de script (à recopier)

```
# -----  
# Titre : <nom du traitement>  
# Objectif : <résumé métier>  
# Entrées : <liste + version>  
# Sorties : <liste + format>  
# Paramètres : <fichier cfg>  
# Historique : <date - auteur - changement>  
# -----
```

12.2. Checklist “prêt pour recette”

11. Le script s'exécute de bout en bout via Rscript sans intervention manuelle.
12. Les dépendances sont figées (renv.lock) et documentées.
13. Les inputs sont validés (présence, schéma, types).
14. Les contrôles QA passent et sont logués (volumétrie, clés, bornes, agrégats).
15. Les outputs sont produits au bon format, au bon emplacement, et versionnés si nécessaire.
16. Les écarts SAS vs R sont documentés et validés par le métier.

Annexe A. Mini-glossaire SAS -> R

- Bibliothèque (libname) : en R, correspond souvent à un dossier (fichiers) ou une connexion base (DBI).
- DATA step : transformations ligne à ligne ; en R, pipelines dplyr/data.table.
- PROC : procédures d'analyse/reporting ; en R, packages spécialisés + Quarto pour le rendu.
- Macro : génération de code / paramétrage ; en R, fonctions + config + itération contrôlée.
- Format : mise en forme / recodage ; en R, dictionnaires + fonctions de recodage.